ON FORMAL VERIFICATION TECHNIQUES FOR MASKED CIRCUITS: APPROACHES AND THEORETICAL UNDERPINNINGS

by
KAĞAN GÜLSÜM

Submitted to the Graduate School of Engineering and Natural Sciences
in partial fulfilment of
the requirements for the degree of Master of Science
in Computer Science and Engineering

Sabancı University
July 2026

ON FORMAL VERIFICATION TECHNIQUES FOR MASKED CIRCUITS: APPROACHES AND THEORETICAL UNDERPINNINGS

Approved by:

Asst. Prof. SÜHA ORHUN MUTLUERGİL .....
(Thesis Supervisor)

Prof. ERKAY SAVAŞ .....

Asst. Prof. KLAUS FREIHERR VON GLEISSENTHAL .....

Date of Approval: July 22, 2026

ABSTRACT

ON FORMAL VERIFICATION TECHNIQUES FOR MASKED CIRCUITS: APPROACHES AND THEORETICAL UNDERPINNINGS

KAĞAN GÜLSÜM

Computer Science and Engineering, M.Sc. Thesis, July 2026

Thesis Supervisor: Asst. Prof. SÜHA ORHUN MUTLUERGİL

Keywords: keyword1, keyword2, keyword3, keyword4

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

ÖZET

MASKELENMIŞ DEVRELER İÇIN FORMAL DOĞRULAMA TEKNIKLERI ÜZERINE: YAKLAŞIMLAR VE KURAMSAL TEMELLER

KAĞAN GÜLSÜM

Bilgisayar Bilimi ve Mühendisliği, Yüksek Lisans Tezi, Temmuz 2026

Tez Danışmanı: Dr. Öğr. Üyesi SÜHA ORHUN MUTLUERGİL

Anahtar Kelimeler: kelime1, kelime2, kelime3, kelime4

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc, Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. et magnis dis parturient montes, nascetur ridiculus mus.

ACKNOWLEDGEMENTS

I would like to thank my advisor and family.

Dedicated to Okan Buruk...

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF SYMBOLS

$\mathbb{F}_2$ The two-element field $\{0,1\}$, with addition $\oplus$ and multiplication $\odot$.

$\oplus$ Addition in $\mathbb{F}_2$ (exclusive-or).

$\odot$ Multiplication in $\mathbb{F}_2$ (logical-and).

$\mathcal{S}$ Set of secret input wires of a circuit.

$\mathcal{R}$ Set of random (masking) input wires of a circuit.

$\mathcal{P}$ Set of public input wires of a circuit.

$\mathcal{W}$ Set of all wires of a circuit.

$\mathrm{vars}(w)$ The random inputs a wire w depends on, $\mathrm{vars}(w) \subseteq \mathcal{R}$.

ρ A randomness assignment, or tape, $\rho : \mathcal{R} \to \{0,1\}$; the randoms are sampled uniformly from the randomness space $\{0,1\}^{\mathcal{R}}$.

$\mathbf{a}, \mathbf{b}$ Secret assignments, $\mathbf{a}, \mathbf{b} \in \{0,1\}^{\mathcal{S}}$.

$\mathbf{w}$ A probe: an observation set of at most d wires, listed as a tuple $\mathbf{w} = (w_1, \ldots, w_k)$.

O An observation set (a probe viewed as an unordered set of wires).

$\mathcal{L}_{\mathbf{a}}(O)$ The law (distribution) of a probe O under secret assignment $\mathbf{a}$, taken over uniform ρ.

d Probing order (maximum number of wires an adversary may observe).

σ Randomness substitution $r \mapsto e \oplus r$; its induced measure-preserving map on the random space is written $\widehat{\sigma}$.

$\varphi_{\mathbf{a}}$ The composite tape map a discharge builds under a single secret assignment $\mathbf{a}$; the coupling is $\Phi_{\mathbf{a},\mathbf{b}} = \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$.

$\mathbf{\Phi}$ Coupling functions; a family $\{\Phi_{\mathbf{a},\mathbf{b}}\}$ of measure-preserving bijections of the random space, one per ordered pair of secret assignments.

#P The counting complexity class.

LIST OF ABBREVIATIONS

ANF Algebraic Normal Form (the unique multilinear $\mathbb{F}_2$ polynomial of a Boolean function).

DAG Directed Acyclic Graph.

ISW Ishai, Sahai, Wagner.

SAT Boolean Satisfiability.

SMT Satisfiability Modulo Theories.

XOR Exclusive-OR, i.e. addition in $\mathbb{F}_2$.

1. INTRODUCTION

1.1 Side-Channel Attacks and the Masking Countermeasure

A cryptographic scheme can be secure on paper and insecure on the bench. When an algorithm runs on a real device its power draw, its timing, and its electromagnetic emanations all vary with the data it handles, and an adversary who measures those physical quantities can recover a key the mathematics was supposed to protect. This is called a *side-channel attack*, and differential power analysis (Kocher et al., 1999) is one type of such an attack. By correlating many power traces against a guess of an intermediate value, it extracts secret keys from implementations whose black-box behaviour is beyond reproach. The insecurity lies not in the cipher but in the way the hardware computes it.

Masking is the standard algorithmic countermeasure (Chari et al., 1999) against differential power analysis. In general it splits each secret value into n shares so that any $n-1$ of them are independent of the secret, and carries the computation out on the shares rather than on the secret itself. The shares are combined by an operation fixed in advance, and the choice of operation depends on the masking scheme. Arithmetic masking adds the shares in a ring and multiplicative masking multiplies them, but the form we adopt is *Boolean masking*, where the operation is exclusive-or over $\mathbb{F}_2$. In a masked circuit, a secret x is then written as $x = x_1 \oplus \cdots \oplus x_n$ with the first $n-1$ shares drawn uniformly at random, so that any $n-1$ of them together reveal nothing about x. The computation runs on the shares, and because any $n-1$ of them are independent of the secret, physical leakage that reflects only some of the shares tells an adversary nothing on its own. To learn anything, the adversary must combine leakage from several shares at once, and each extra share raises the measurement cost sharply. Realising this masking across a whole cipher is delicate, since the non-linear operations of the (unmasked) algorithm force the

shares of different variables to interact in the masked circuit, and it is precisely those interactions that a masking scheme must handle with care (Ishai et al., 2003; Rivain & Prouff, 2010).

1.2 Verifying Masked Implementations

Masking is easy to get subtly wrong. Splitting a secret into shares helps only as long as the shares are never brought back together somewhere an attacker can reach, and a real implementation gives them many chances to. A single wire may recombine two shares of the same secret, or a random mask meant to hide one value may cancel against a copy of itself elsewhere, and either slip reopens the very leak the masking was meant to close. There is a further subtlety. A masking must hold up against an attacker who combines the leakage of several intermediate values at once, not just one, so its correctness cannot be judged value by value; it depends on how the values leak jointly. The number of such combinations grows quickly with the size of the circuit and the number of shares, which puts checking by hand out of reach for all but the smallest implementations.

What one wants, then, is a tool that takes a masked circuit and decides, mechanically and for all of those combinations at once, whether any of them leaks. This is the problem of *formally verifying* a masked implementation, and a line of work has grown around it (Barthe et al., 2015, 2019). The condition to establish is one about distributions, that the values on every small set of wires are distributed the same way whatever the secret is, so that nothing about the secret can be recovered from them. A verifier must certify this for all such sets together, and two difficulties sit underneath. Even one set is already a statement about a distribution over all the internal random choices, and there are combinatorially many sets to settle. The tools that scale do not compute those distributions head-on. They rewrite each set of values into a form in which the secrets have visibly cancelled, and they organise the search so that a single such certificate settles many sets at once. This thesis builds a verifier in that constructive spirit.

1.3 Scope and Threat Model

We work in the *probing model* of Ishai, Sahai, and Wagner (Ishai et al., 2003). The circuit is arithmetic over $\mathbb{F}_2$, built from two-input XOR and AND gates, and its inputs are partitioned into secret, random, and public wires. An adversary of order d chooses any d wires and learns their joint values over the draw of the internal randomness, and the circuit is *d-probing secure* when no such choice tells the secrets apart (Chapter 2). The leakage is *value-based*: a probed wire leaks the logical value it carries and nothing more. Physical effects that make a wire leak about more than its final value, such as glitches and transitions in hardware (Barthe et al., 2019), are outside our model, as is any question of composing separately verified gadgets into a larger one. The object of study is the exact probing security of a single circuit, decided over its wire values.

Within that model our aim is exact verification rather than a sound approximation. When the tool reports a circuit secure, every one of its $\sum_{i \le d} \binom{|\mathcal{W}|}{i}$ observation sets has been accounted for, and when it reports a circuit insecure it returns a specific leaking observation as a witness, up to the one incompleteness we flag in Chapter 5. The decision problem underlying a single observation is hard in general, in a sense Chapter 4 makes precise, which is what forces the constructive route the tool takes rather than a direct count.

1.4 Contributions

This thesis presents Coppula, a verifier of d-probing security for masked arithmetic circuits over $\mathbb{F}_2$, implemented in Lean 4. Its design is organised around three components, and the contributions follow that division.

A constructive oracle for one observation. We give a rewrite engine that decides secret-independence of a single probe by substituting away its randoms one at a time, driving the observation toward a secret-free form (Chapter 5). The engine is layered, a cheap single-occurrence rule first and a complete substitution loop behind it, and we prove its verdicts sound. To reach randoms that ordinary rewriting cannot, we add a *multiplicative factoring* transformation that reshapes

products to expose masks buried inside them, and we show it strictly enlarges the class of circuits the engine can certify.

An exhaustive traversal of the observation space. We give a driver that certifies every d-subset of the wires without enumerating them one by one, splitting the space recursively in the manner of `maskVerif` (Barthe et al., 2019) and proving the split exhaustive through a set-level reading of Vandermonde's identity (Chapter 5). One discharge of a large union can prune a whole branch at once.

A coupling view of rewriting, and probe-set extension. We read a successful discharge not merely as a yes-or-no verdict but as a measure-preserving *coupling* of the randomness, in the tradition of coupling proofs for probabilistic programs (Barthe et al., 2017), developed for probing security in Chapter 4. That reading lets one certificate blind many wires at once. We use it to grow a certified probe into a whole *safe set* of wires, so a single discharge settles many observations, and we compare three mechanisms for the growth, adopting the coupling-based one and proving the other two sit inside it (Chapter 5).

Implementation and evaluation. We describe the data structures that make the engine efficient, a hash-consed circuit representation and a bit-sliced, staged test for the extension (Chapter 6), and we evaluate Coppula on a suite of masked gadgets, reporting its verification outcomes and analysing where the search does redundant work (Chapter 7). Along the way the tool rediscovers a genuine flaw in a four-share gadget, which we use as a case study.

1.5 Thesis Outline

Chapter 2 fixes notation and defines arithmetic circuits, the probing model, and d-probing security, and studies the randomness substitution technique we use througout. Chapter 3 surveys the verification literature we build on and depart from. Chapter 4 develops the coupling view of probing security and situates the underlying decision problem in the complexity hierarchy. Chapter 5 is the core of the thesis, presenting our algorithm with its three components together with their algorithms and correctness proofs, while Chapter 6 gives the concrete data structures and implementation details behind them. Chapter 7 reports the experiments. Chapter 8 outlines the directions our approach does not treat, and Chapter 9 concludes

the thesis.

2. PRELIMINARIES

2.1 Notation and Mathematical Background

This chapter collects the background the rest of the thesis builds on and fixes the notation for it. We write $[n] = \{1, \ldots, n\}$ and use $\{0,1\}$ or $\mathbb{F}_2$ for the set of bits. Tuples are set in boldface, $\mathbf{x} = (x_1, \ldots, x_n)$, with $|\mathbf{x}|$ denoting their length. For a finite set X we write $\mathcal{P}(X)$ for its power set and $\binom{X}{k}$ for the family of its k-element subsets. Functions are total unless we say otherwise, and we identify a function $\{0,1\}^n \to \{0,1\}$ with the Boolean function it computes. Notation that is used only locally is introduced where it appears.

2.1.1 The Field $\mathbb{F}_2$ and Multilinear Boolean Polynomials

The field $\mathbb{F}_2$ has carrier $\{0,1\}$ with addition $\oplus$, the exclusive-or, and multiplication $\odot$, the logical-and. Addition is its own inverse, so $x \oplus x = 0$, and multiplication is idempotent, $x \odot x = x$.

Every function $f : \mathbb{F}_2^n \to \mathbb{F}_2$ is a polynomial over $\mathbb{F}_2$, and idempotence lets us take that polynomial to have no variable raised above the first power. So f has a unique *multilinear* representation,

$$f(x_1, \ldots, x_n) \;=\; \bigoplus_{S \subseteq [n]} c_S \bigodot_{i \in S} x_i, \qquad c_S \in \mathbb{F}_2,$$

its *algebraic normal form* (ANF). Uniqueness is what turns the ANF into a decision procedure for the questions we put to it. In particular, note that, f depends on

a variable x_i (meaning that flipping x_i can change the output) if and only if some monomial of the ANF contains x_i.

2.1.2 Probability Distributions and Statistical Independence

Probing security is a statement about distributions, so we fix the little we need. A *distribution* on a finite set X is a function $\mu : X \to [0,1]$ with $\sum_{x \in X} \mu(x) = 1$, and the *uniform* distribution assigns $\mu(x) = 1/|X|$ to every x. A random variable Z into a finite set induces the distribution $x \mapsto \Pr[Z = x]$, its *law*, and two random variables are *identically distributed* when their laws agree on every value.

The security notion we are after is the independence of a family of laws from a parameter. Suppose a random variable Z depends on a parameter s, so that each value of s gives a law μ_s. We say Z is *independent of* s when μ_s is one and the same distribution for every value of s. For probing security the parameter is the secret assignment and the law is that of a probe, as Section 2.3.2 makes precise. One way to prove that two laws are equal is through a measure-preserving bijection: if π carries the uniform distribution on X to itself, then for any function g the variables $g(Z)$ and $g(\pi(Z))$ are identically distributed when Z is uniform.

2.1.3 Boolean Circuits and Gates

We now introduce the object the rest of the thesis is built on. The definition is the standard digraph formulation of a circuit, specialised to arithmetic over the two-element field.

Definition 2.1 (Arithmetic circuit over $\mathbb{F}_2$)**.** *Fix the field $\mathbb{F}_2 = \{0,1\}$ and a set $S = \{s_i\}$ of* gate functions*, each of some arity $a_i \in \mathbb{N}_{\geq 1}$, i.e. each s_i maps $\mathbb{F}_2^{a_i}$ to $\mathbb{F}_2$. A* circuit *C is a finite directed graph (a* digraph*) containing no directed cycle, such that:*

- *Every vertex of indegree 0 is labelled either by an* indeterminate*, in which case it is an* input gate*, or by an element of $\mathbb{F}_2$, in which case it is a* constant gate*. The input gates are partitioned into* secret *inputs $\mathcal{S}$,* random *inputs $\mathcal{R}$, and* public *inputs $\mathcal{P}$.*

- *Every remaining vertex is a* computation gate*, labelled by a function $s_i \in S$ whose arity a_i coincides with the vertex's indegree. A computation gate of outdegree* 0 *is an* output gate*.*

We call the vertices wires*, and for a computation gate we speak interchangeably of the gate and the wire carrying its output.*

Over $\mathbb{F}_2$ there is no difference between the Boolean and the arithmetic reading of such a circuit: exclusive-or is addition, logical-and is multiplication, and the two together already express every function $\mathbb{F}_2^n \to \mathbb{F}_2$, i.e. they are universal. So we fix

$$S = \{\oplus,\ \odot\},$$

addition and multiplication in $\mathbb{F}_2$ (Section 2.1.1), both of arity 2. Every computation gate then has indegree exactly 2 and reads

$$w_i = w_j \oplus w_k \qquad \text{or} \qquad w_i = w_j \odot w_k.$$

Throughout the thesis $\odot$ denotes multiplication in $\mathbb{F}_2$; we might sometimes use $\cdot$ for the same operation.

Acyclicity is what makes a circuit computable. It lets us list the wires in a topological order $w_1, \ldots, w_m$ in which the two operands of every gate appear before the gate itself, so that $j, k < i$. Fixing a value in $\mathbb{F}_2$ for every input gate — a constant gate carries its own label — then determines the value of every wire by induction along this order. When the inputs are clear from context we write simply w_i for this value.

Notation 2.1. *For a wire w_i,* $\mathrm{vars}(w_i) \subseteq \mathcal{R}$ *is the set of random inputs its defining sub-circuit depends on:* $\mathrm{vars}(w_i) = \{w_i\}$ *if $w_i \in \mathcal{R}$,* $\mathrm{vars}(w_i) = \emptyset$ *if $w_i \in \mathcal{S} \cup \mathcal{P}$ or w_i is a constant gate, and* $\mathrm{vars}(w_i) = \mathrm{vars}(w_j) \cup \mathrm{vars}(w_k)$ *for a gate with operands w_j, w_k.*

Two conventions, fixed here once, are used throughout. First, a topological order realises the circuit as a *straight-line program*: each wire is defined at most once, in terms only of wires already defined, so there is no branching and no recomputation of a gate under a different name. We pass freely between the digraph of Definition 2.1 and this sequential reading, whichever is clearer at the moment. Second, the restriction to 2-ary gates is a modelling choice, not a loss of generality: an n-ary sum or product is always expressible as a chain of 2-ary gates.

2.2 Secret Sharing and Masking

Physical implementations of cryptographic hardware can leak information. Power draw, electromagnetic emanation, and timing all correlate with the values a device computes on, and an attacker who measures them can recover a key that is out of cryptographic reach (Kocher et al., 1999). *Masking* is the standard algorithmic countermeasure (Chari et al., 1999). Instead of computing on a secret directly, one splits it into several *shares* whose joint distribution hides it, then rewrites the computation to run on the shares throughout and reconstruct only at the very end. The promise is that an attacker who sees few enough intermediate values learns nothing, because too few shares carry no information about the secret.

2.2.1 Boolean Masking

We use *Boolean* masking, where shares combine by exclusive-or. An *n-sharing* of a bit $x \in \mathbb{F}_2$ is a tuple $(x_0, \ldots, x_{n-1})$ with $x_0 \oplus \cdots \oplus x_{n-1} = x$. To draw one, sample $x_1, \ldots, x_{n-1}$ uniformly and independently and set $x_0 = x \oplus x_1 \oplus \cdots \oplus x_{n-1}$. The construction is built so that any $n-1$ of the shares are then uniform and independent, hence reveal nothing about x; only the full set of n shares determines it. This is the same statement, for one bit, that Section 2.1.2 calls independence of the secret.

A masked circuit carries every value in shared form and replaces each operation by a *gadget*, a sub-circuit computing a sharing of the result from sharings of its inputs. Linear operations are trivial, since $\oplus$ acts share by share. Multiplication is where the difficulty and the fresh randomness live, as a gadget for it must combine shares of both inputs without ever forming a value that depends on a secret. The masked AND of Ishai, Sahai and Wagner (Ishai et al., 2003) is the archetype, and the circuits we verify are built from gadgets of this kind.

2.3 The ISW Probing Model

For the security notion to be precise we need to state an adversary. Ishai, Sahai and Wagner (Ishai et al., 2003) gave the model the thesis works in: the probing model. An adversary places a bounded number of probes on the wires of a circuit and reads the values they carry, and the circuit is secure when no choice of probes tells the adversary anything about the secrets. The model abstracts a physical side-channel attacker into a clean combinatorial game, and it is the setting in which masking admits proofs.

The connections between the physical side-channel attack and the probing model, however, are not exposited in this thesis.

2.3.1 Threshold Probing and Observation Sets

Fix a circuit with secret inputs $\mathcal{S}$, random inputs $\mathcal{R}$, and public inputs $\mathcal{P}$ as in Section 2.1.3. A *threshold* adversary of order d chooses an *observation set* of at most d wires and learns their joint values. The public inputs are known and the secrets are held fixed, so the only randomness in play is the internal randomness $\mathcal{R}$, sampled uniformly. An observation set therefore carries a joint distribution over the draw of $\rho \in \{0,1\}^{\mathcal{R}}$. We call such a set a *probe*, and we read each of its wires as a function of ρ.

2.3.2 d-Probing Security

Definition 2.2 (d-probing security)**.** *A circuit is* d-probing secure *if for every observation set of at most* d *wires the joint distribution of that set, taken over uniform* $\rho \in \{0,1\}^{\mathcal{R}}$*, is independent of the secret inputs in the sense of Section 2.1.2. Equivalently, the law of the set is one and the same for every assignment to* $\mathcal{S}$*.*

The order d counts the probes the adversary is granted, and an n-share masking is usually meant to withstand $d = n - 1$ of them. The definition quantifies over sets of

wires read from a single run of the circuit, rather than single indivdual wires, since a masking leaks only jointly. And it is a worst-case statement over all $\sum_{i \le d} \binom{|\mathcal{W}|}{i}$ observation sets of the circuit's wires $\mathcal{W}$. Verifying this definition for the circuits we defined is the task of the whole thesis.

2.3.3 Probabilistic Non-Interference

The distributional condition of Definition 2.2 has an equivalent phrasing the verification literature prefers, *probabilistic non-interference* (Barthe et al., 2015). It helps to study it and understand in different views, so below we give three perspectives that realize the same notion. Fix a probe O with $|O| \le d$. Under a secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$ and uniform randomness ρ, the probe takes the joint value $O(\rho;\mathbf{a}) = \big(w(\rho;\mathbf{a})\big)_{w \in O}$ and we write $\mathcal{L}_{\mathbf{a}}(O)$ for its law — the distribution of that value over the draw of ρ.

The first reading is an equality of laws. Definition 2.2 says of a d-probing secure circuit that

$$\mathcal{L}_{\mathbf{a}}(O) = \mathcal{L}_{\mathbf{b}}(O) \qquad \text{for all } \mathbf{a}, \mathbf{b} \in \{0,1\}^{\mathcal{S}} \text{ and every } O \text{ with } |O| \le d,$$

so the law of a probe is one and the same whatever the secrets are, hence we cannot deduce information about the secrets by looking at the values on the probe.

The second reading is *simulability*, and it is the one that names non-interference. A probe O is non-interfering when some randomised algorithm $\mathsf{Sim}(O)$, given the probe and the public inputs but never the secrets, has output law equal to $\mathcal{L}_{\mathbf{a}}(O)$ for every $\mathbf{a}$. Such a probe shows the adversary nothing it could not have produced on its own without the secret, which is the guarantee masking is meant to provide. Simulability and equality of laws are the same property. If the laws agree, their common value is a distribution the simulator samples outright, and conversely a single simulator that matches every $\mathbf{a}$ forces all the $\mathcal{L}_{\mathbf{a}}(O)$ to coincide with its output, hence with one another.

The third reading is a game, and it is the sharpest picture of what security denies the adversary. The adversary names a probe O of size at most d together with two secret assignments $\mathbf{a}_0$ and $\mathbf{a}_1$. A challenger draws a hidden bit $c \in \{0,1\}$ and fresh randomness ρ, runs the circuit under $\mathbf{a}_c$, and hands back the transcript $O(\rho;\mathbf{a}_c)$. The

adversary guesses c, and its advantage $\left|\Pr[\text{guess} = c] - \frac{1}{2}\right|$ is at most the statistical distance

$$\tfrac{1}{2}\sum_{o}\left|\mathcal{L}_{\mathbf{a}_0}(O)(o) - \mathcal{L}_{\mathbf{a}_1}(O)(o)\right|$$

between the two laws, with equality for the optimal guess. The circuit is d-probing secure exactly when this distance is zero for every O, $\mathbf{a}_0$, and $\mathbf{a}_1$, so that the two secret worlds are perfectly indistinguishable through any d wires. We call this *secret-indistinguishability*.

The three are one condition seen from three sides, equal laws, a secret-free simulator, and a game no adversary wins. Non-interference is the form the verification literature builds on, because it turns a question about distributions into one about expressions.

2.4 Randomness Substitution

This section isolates, as a single reusable fact, the probabilistic argument our checker automates. Fix a circuit as in Section 2.1.3, with random inputs $\mathcal{R}$, and let a *randomness assignment*, or *tape*, be a function $\rho : \mathcal{R} \to \{0,1\}$. The secrets and public inputs are held fixed throughout, so every wire w denotes a function $w(\rho)$ of ρ alone. A *probe* is an observation set of Section 2.3.2, which we list as a tuple $\mathbf{w}$ of wires; its order does not affect the joint distribution. That section phrases d-probing security as the demand that the joint distribution of a probe of size at most d, taken over the uniformly distributed ρ, not move as the secrets vary.

The whole argument rests on one line of $\mathbb{F}_2$ arithmetic: for any fixed bit c, the translation/shear $b \mapsto c \oplus b$ is a **bijection** of $\{0,1\}$ that preserves the uniform distribution (i.e. it is its own inverse and merely swaps the two outcomes). Everything below is this fact, applied to a single coordinate of ρ with every other coordinate fixed.

Definition 2.3 (Randomness substitution)**.** *Let $r \in \mathcal{R}$ and let e be a wire with $r \notin \mathrm{vars}(e)$. The* substitution $\sigma = (r \mapsto e \oplus r)$ *acts on a wire w by replacing every occurrence of the input r with the wire $e \oplus r$; we write the resulting wire as $w\sigma$. The same σ acts on randomness assignments, through the map $\widehat{\sigma} : \{0,1\}^{\mathcal{R}} \to \{0,1\}^{\mathcal{R}}$ whose image $\rho' = \widehat{\sigma}(\rho)$ shifts the r-coordinate by $e(\rho)$ and leaves every other coordinate untouched:*

$$\rho'(r) = e(\rho) \oplus \rho(r), \qquad \rho'(r') = \rho(r') \quad \text{for } r' \neq r.$$

Geometrically, $\widehat{\sigma}$ is a *shear* of the cube $\{0,1\}^{\mathcal{R}}$: it fixes every coordinate but r and offsets that one by $e(\rho)$. Restricted to a fibre it is exactly the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$ of the one-line fact above, so what measure it preserves it preserves by translation invariance. We call any such map, one coordinate translated by a context depending on the others, and everything else left intact, a *shear*, and use the word throughout.

We pair the two actions under one name with the lemma below. The syntactic rewrite $w \mapsto w\sigma$ and the reparametrisation $\rho \mapsto \widehat{\sigma}(\rho)$ are one operation seen from two sides, and $\widehat{\sigma}$ does not disturb the distribution.

Lemma 2.1 (Substitution)**.** *For a substitution $\sigma = (r \mapsto e \oplus r)$ as in Definition 2.3, the map $\widehat{\sigma}$ is a measure-preserving involution of the uniform distribution on $\{0,1\}^{\mathcal{R}}$, and for every wire w,*

$$(w\sigma)(\rho) \;=\; w\Big(\widehat{\sigma}(\rho)\Big) \qquad \textit{for every randomness assignment } \rho.$$

Proof. Fix every coordinate of ρ other than r. Since $r \notin \mathrm{vars}(e)$, the value $e(\rho)$ depends only on these fixed coordinates, so it is constant as $\rho(r)$ ranges over $\{0,1\}$. On this fibre $\widehat{\sigma}$ acts by the translation $\rho(r) \mapsto e(\rho) \oplus \rho(r)$, which is measure-preserving. Hence $\widehat{\sigma}$ is a measure-preserving involution of $\{0,1\}^{\mathcal{R}}$ preserving the uniform measure coordinatewise, and so preserving the product measure.

To prove the stated identity, do a structural induction on w. For $w = r$ both sides equal $e(\rho) \oplus \rho(r)$. For an input $w \neq r$ both sides equal $w(\rho)$, since $\widehat{\sigma}$ fixes every coordinate but r, they are also equal. For a gate $w_i = w_j \oplus w_k$ or $w_i = w_j \odot w_k$, just apply the hypothesis to w_j and w_k and use that $\oplus$ and $\odot$ are computed pointwise from their operands' values. □

Corollary 2.1 (Single-occurrence discharge)**.** *Suppose r occurs exactly once among a probe $\mathbf{w}$ and the wires it depends on, as an operand of a gate $x = e \oplus r$ with $r \notin \mathrm{vars}(e)$, and let $\sigma = (r \mapsto e \oplus r)$. Then $\mathbf{w}\sigma$ is identically distributed to $\mathbf{w}$ over ρ uniform, and every occurrence of x in $\mathbf{w}\sigma$ has collapsed to the bare input r.*

Proof. Distributional equality is Lemma 2.1 applied coordinatewise to $\mathbf{w}$, using that $\widehat{\sigma}$ preserves the uniform distribution on $\{0,1\}^{\mathcal{R}}$. For the second claim, $x\sigma = e \oplus (e \oplus r) = r$ by $\mathbb{F}_2$-cancellation; since r occurs nowhere else by hypothesis, no other part of $\mathbf{w}$ is touched. □

Remark 2.1. *Rewriting a probe by a substitution — whether in the general form of Lemma 2.1 or the single-occurrence case of Corollary 2.1 — leaves its distribution*

untouched. That is the key observation that derives the procedure to decide the security of a probe, by one substitution at a time, taking it to a form with no secret in sight. The single-occurrence case is the one the `maskVerif` *tool (Barthe et al., 2019) calls* optimistic sampling. *A derivation is nothing more than such substitutions chained together; how that chain is searched for is taken up in the later chapters.*

2.5 Computational Hardness Preview

Definition 2.2 asks, for each observation set, whether a distribution stays fixed as the secrets vary, and read literally this is a counting question, beacuse the measures we work with are defined on discrete spaces endowed with the counting measure. The law of a probe assigns to each possible observation the number of randomness assignments ρ that produce it, divided by $2^{|\mathcal{R}|}$, and independence of the secrets asks that these counts agree across secret settings. Chapter 4 makes the difficulty precise, showing that the associated counting problem is #P-complete, #P being the class Valiant introduced for counting the accepting computations of a nondeterministic machine (Valiant, 1979).

The verification question of d-probing security, however, is a *decision* problem rather than a counting one, so it cannot be a member of #P. Deciding whether one probe is independent of the secrets compares two counts and asks whether they are equal, and the class of decision problems of this shape, i.e. whether a nondeterministic machine has exactly a prescribed number of accepting paths, is $\mathrm{C}_{=}\mathrm{P}$, the exact-counting counterpart of #P studied by Wagner (Wagner, 1986). Its canonical complete problem, deciding whether a Boolean formula has exactly a given number of satisfying assignments, is the equality-of-counts question in its essence, and probing security is that same question, but posed in the language of masked circuits. Locating the problem in $\mathrm{C}_{=}\mathrm{P}$ says plainly that an exact decision procedure is not expected to be efficient at all, and it is the reason this thesis does not count. It certifies security constructively instead, through the rewriting of Section 2.4, giving a checker that is sound but necessarily incomplete.

3. RELATED WORK

3.1 Language-Based Verification

Survey language-based approaches to masking verification, the main inspiration for this thesis.

3.1.1 Verified Proofs of Higher-Order Masking

Discuss prior work on verified/formal proofs of higher-order masking schemes.

3.1.2 Large Sets and the Exploration Algorithm

Discuss prior large-set exploration algorithms for probing security and how this thesis's subset-space traversal relates.

3.1.3 Witness Replay and Derivation Reuse

Discuss prior work on reusing a certification witness/derivation across related probes, and how it relates to Component III's replay-driven extension (Section 5.4.2).

3.1.4 MaskVerif With Physical Defaults

Discuss `maskVerif` and its physical-default extensions, and compare to the rewrite engine of Chapter 5.

3.2 Fourier-Analytic Verification

Survey Fourier-analytic approaches to masking verification (Bloem et al.).

3.2.1 The Xiao-Massey Criterion and Approximations

Discuss the Xiao-Massey criterion and its approximate variants for testing correlation-immunity/independence.

3.2.2 SAT-Based Realization and Its Cost

Discuss SAT-based realizations of Fourier-analytic verification and their computational cost.

3.3 Other Approaches

Survey other verification approaches not covered above.

3.3.1 Model Counting and SMT

Discuss model-counting and SMT-based approaches to verifying secret-independence.

3.3.2 Compositional Reasoning and Relabeling

Discuss compositional reasoning and relabeling/symmetry techniques used elsewhere, relating to Future Work's symmetry-orbit proposal.

3.4 Position of This Work

Position this thesis's contributions relative to the surveyed related work.

4. COUPLINGS AND PROBING SECURITY

4.1 The Counting Problem Is #P-Complete

Introduce the counting problem behind secret-independence checking and state the #P-completeness result.

4.1.1 From Independence to Distribution Counting

Show how deciding secret-independence of a probe reduces to a distribution-counting problem.

4.1.2 #P-Completeness and Its Consequences for Verification

Prove #P-completeness of the counting problem and discuss what this implies for exact verification (motivating the constructive rewrite-based approach instead).

4.2 Couplings of Probabilistic Programs

Introduce couplings as a proof technique for distributional equality between probabilistic programs.

4.2.1 Coupling Two Runs Under Different Secrets

Define a coupling between the run of a probe under one secret assignment and its run under another.

4.2.2 Coupling Functions: Randoms to Randoms

Formalize coupling functions as measure-preserving bijections on the randomness domain.

4.3 Probing Security via Couplings

Relate d-probing security to the existence of a coupling for every probe of size d.

4.3.1 d-Probing Security Implies a (Family of) Coupling Functions

Prove that d-probing security is equivalent to the existence of a family of coupling functions, one per pair of secret assignments.

4.3.2 Secret-Indistinguishability

Connect the coupling-existence characterization back to the standard secret-indistinguishability definition of probing security.

4.4 Substitutions as a Coupling Construction

Explain how the randomness-substitution rewrites of Section 2.4 construct an explicit coupling.

4.4.1 The Derivation of an Explicit Coupling Witness

Show how a derivation (sequence of substitutions) yields an explicit coupling witness for a probe.

4.4.2 Why a Successful Rewrite Yields a Valid Coupling

Prove that a successful rewrite derivation, read as a composition of shears, satisfies the coupling definition.

5. DESIGN OF COPPULA

5.1 Pipeline Overview

Coppula takes an arithmetic circuit over $\mathbb{F}_2$, where the wires are partitioned into secret, random, and public inputs (Chapter 2), together with a probing order d, and returns a verdict: the circuit is d-probing secure, or it is not and a witnessing probe is handed back. Underneath sits the single question of Section 2.3.2, is every set of at most d wires secret-independent, and two difficulties have to be resolved for an answer. Deciding one single probe is already non-trivial — it is a statement about a distribution over the randomness space — and there are combinatorially many probes to decide. The design addresses both difficulties.

Three components divide the labour. *Component I*, the rewrite engine (Section 5.2), is the oracle for a single probe. It decides secret-independence not by summing over a distribution but by rewriting the probe's expression, one randomness substitution at a time, into a form with no secret in sight (Section 2.4). *Component II*, the subset-space traversal (Section 5.3), is the driver. It sweeps every d-subset of the wires without enumerating them one by one, in the recursive set-splitting style of `maskVerif` (Barthe et al., 2019). *Component III*, the probe-set extension (Section 5.4), is the amplifier. It turns the certificate for one probe into a certificate for a strictly larger safe set of wires, so that a single discharge settles many probes at once.

All three components someway use couplings. When Component II reaches a batch of probes it cannot dismiss as a whole, it asks Component I to certify a representative; a successful discharge returns not just a secure verdict but a measure-preserving relabelling of the randomness space (a coupling function) that witnesses it. Component III substitutes that coupling in the circuit's other wires and gathers those that

are blinded (rid of secrets) into the safe set. Component II then splits the batch about that safe set and recurses, with the safe part already certified. Soundness of the whole rests on Component I alone (Theorem 5.1); Components II and III only utilize its verdicts, adding neither unsoundness nor any incompleteness of their own.

The rest of the chapter takes the three components in turn, asking what each computes and why it is correct. The concrete data structures that make them time- and memory-wise efficient are explained in Chapter 6.

5.2 Component I: The Rewrite Engine

Given a probe $\mathbf{w} = (w_1, \ldots, w_k)$, $k \leq d$ — a tuple of wires of the circuit — the rewrite engine decides whether $\mathbf{w}$ is secret-independent. It does so constructively, rather than reasoning about probability distributions directly; it searches for a finite sequence of applications of Lemma 2.1 (Section 2.4) after which $\mathbf{w}$'s form is syntactically free of secrets — no secret is present in the expression of any w_i. Termination with a secret-free image certifies $\mathbf{w}$. Theorem 5.1 in Section 5.2.3 makes precise why this certificate is sound.

The engine has three tiers of rewriting rules, increasing in generality and cost, summarized in Algorithm 1 and mirrored in the structure of this section. SimpleRewrite (Section 5.2.1) handles the common case of every random occurring at most once, cheaply. It is tried first on the unfactored probe and, on failure, once more after the multiplicative factoring of Section 6.1.3 has had a chance to expose additional single-occurrence randoms. This factoring & rewriting is the second tier. GeneralRewrite (Section 5.2.2) is the complete fallback for the remaining case, for random variables occurring more than once. It substitutes a random by its context wherever the random occurs, interleaving factoring lazily whenever both of its own find-rules stall. Section 5.2.3 closes the section by reading the whole derivation as an explicit, replayable bijection on the randomness space. Which becomes the Φ that Component III (Section 5.4) later utilizes in every other wire in the circuit.

Algorithm 1 Discharging a probe (Component I)

function CheckProbe($\mathbf{w}$)
Require: probe $\mathbf{w}$, a tuple of wires of the circuit
Ensure: Secure, or Insecure($\mathbf{w}$)
    $(ok, T) \leftarrow$ SimpleRewrite($\mathbf{w}$) ▷ Tier 1: unfactored
    if *ok* **then return** Secure
    end if
    $(ok, T) \leftarrow$ SimpleRewrite(Factor($\mathbf{w}$)) ▷ Tier 2: factor (§6.1.3), retry
    if *ok* **then return** Secure
    end if
    $(ok, T) \leftarrow$ GeneralRewrite($\mathbf{w}, \emptyset, \mathit{fuel}, \mathit{false}, \langle\rangle$) ▷ Tier 3: complete fallback
    return *ok* ? Secure : Insecure($\mathbf{w}$)
end function

function SimpleRewrite($\mathbf{w}$)
    $\mathit{todo} \leftarrow \{ r \in \mathrm{vars}(\mathbf{w}) : r$ occurs once, additively, $r \notin \mathbf{w} \}$
    (sort todo w.r.t. increasing multiplicative depth)
    $T \leftarrow \langle\rangle$
    while $\mathit{todo} \neq \langle\rangle$ **and not** SecretFree($\mathbf{w}$) **do**
        $r \leftarrow$ pop shallowest element of *todo*
        $x \leftarrow$ unique additive parent of r ▷ $x = e \oplus r$
        mark x as a fresh random leaf
        $T \leftarrow T.\mathrm{push}(r, x)$
        drop r
        (a random now matching the pattern above, $\notin \mathbf{w}$, may join todo)
    end while
    return (SecretFree($\mathbf{w}$), T) ▷ shears now recorded — §5.2.3
end function

function GeneralRewrite($\mathbf{w}, \mathit{used}, \mathit{fuel}, \mathit{factored}, T$)
    if SecretFree($\mathbf{w}$) **then return** (true, T)
    end if
    if $\mathit{fuel} = 0$ **then return** (false, T)
    end if
    $r \leftarrow$ FindSimple($\mathbf{w}$) ▷ single-occurrence, non-root, additive
    if $r = \perp$ **then** $r \leftarrow$ FindGeneral($\mathbf{w}, \mathit{used}$) ▷ multi-occurrence, at least one additive instance
    end if
    if $r = \perp$ **then**
        if *factored* **then return** (false, T)
        end if
        return GeneralRewrite(Factor($\mathbf{w}$), *used*, *fuel*, *true*, T) ▷ lazy factoring
    end if
    $x \leftarrow$ chosen additive parent of r; $e \leftarrow \bigoplus(\mathrm{children}(x) \setminus \{r\})$ ▷ $r \notin \mathrm{vars}(e)$
    $t \leftarrow e \oplus r$; $\mathbf{w}' \leftarrow \mathbf{w}[r \mapsto t]$ ▷ Lemma 2.1
    return GeneralRewrite($\mathbf{w}', \mathit{used} \cup \{r\}, \mathit{fuel} - 1, \mathit{false}, T.\mathrm{push}(r, t)$)
end function

5.2.1 The Simple Rewrite Rule

The cheapest tier discharges the pattern Corollary 2.1 addresses directly, where a random occurs exactly once below the probe, additively (as an operand of an XOR gate). Such a random is a simple-rule candidate, provided it is not itself a probe root. Writing $x = e \oplus r$ for its unique parent, the corollary lets us relabel x as a fresh random and drop r without ever looking at the context e. The pattern certifies that x, jointly with everything independent of r, is distributed as a fresh, independent randomness. The tier applies this rewrite repeatedly, and with each rewrite it possibly creates further randoms fitting the pattern, until either no secret variable is reachable from the probe (success) or no candidate remains.

The exclusion of probe roots is a soundness condition. A root's value is what the adversary observes, so it is fixed, and relabelling it would claim independence for something the probe actually exposes.

The order in which rewrites fire is based on the multiplicative depth of the random variable. Randoms closest to a root get rewritten first.

This tier is sound, since each step is a direct instance of Corollary 2.1, but it is incomplete. It has nothing to offer for a random occurring twice, such as a mask reused across two cross-terms of a masked multiplication. Section 5.2.2's fallback handles such cases.

5.2.2 Fallback: General Substitution

GeneralRewrite handles randoms that occur more than once by performing the substitution of Lemma 2.1 literally, rather than only its single-occurrence case (Corollary 2.1). It picks a random r and an additive occurrence $x_1 = e \oplus r$ with $r \notin \mathrm{vars}(e)$, then rewrites every root by substituting $r \mapsto e \oplus r$ throughout. Cancellation in $\mathbb{F}_2$ collapses the chosen occurrence x_1 to the bare leaf r, exactly as in Corollary 2.1, while every other occurrence of r absorbs e instead of vanishing, cancelling further if it happens to share structure with e. The loop succeeds once no secret variable remains in the rewritten roots.

Each round re-examines the current probe for an applicable substitution. FindSimple looks first for the same single-occurrence, non-root pattern as Section 5.2.1, since

a general substitution can turn a multi-occurrence random into a single-occurrence one; it prefers the shallowest candidate. Failing that, FindGeneral looks for any random, not yet used by a previous general step, occurring at least twice with some additive parent x_1 whose other children do not themselves depend on it — the side condition $r \notin \mathrm{vars}(e)$ that Lemma 2.1 requires.

When both find-rules stall on the probe's current form, the engine factors (Section 6.1.3) once and retries, rather than declaring failure immediately. Factoring can move a random from a purely multiplicative position, where neither find-rule can see it, into an additive one, but on its own it eliminates no secret, so it must be interleaved with substitution rather than run once up front or once at the end. Factoring is symmetrically not tried before the simple rule either. Algorithm 1 always tries SimpleRewrite unfactored first, because factoring can destroy a single-occurrence opportunity the unfactored form exposes, so trying the cheap, unfactored tier first is not just faster on the common case but strictly more complete.

FindGeneral is needed precisely when a random occurs additively in two places that do not telescope into one another under the simple rule. Suppose r occurs as an operand of $x_1 = e_1 \oplus r$ in one root and as $x_2 = e_2 \oplus r$ in another, with $r \notin \mathrm{vars}(e_1) \cup \mathrm{vars}(e_2)$. Substituting through x_1, i.e. $r \mapsto e_1 \oplus r$, turns x_1 into $e_1 \oplus (e_1 \oplus r) = r$ by cancellation, as in Corollary 2.1, while the same substitution turns x_2 into $e_2 \oplus e_1 \oplus r$. If $e_1 \oplus e_2$ is itself secret-free, both roots are secret-free after this one step; otherwise the loop continues, substituting further or factoring to expose a new opportunity. This is the shape of the case a masked multiplication typically produces, where a single random masks two different cross-terms: neither term's random is disposable on its own, but the substitution above can still cancel the secret-dependent part they share. Whether a given instance is discharged this way, and in how many rounds, depends entirely on how much of $e_1 \oplus e_2$ the rest of the derivation can still cancel; the loop offers no guarantee beyond trying. Even granting unlimited fuel, it only composes single-coordinate shears $\widehat{\sigma}$, so it can only certify secret-independence witnessed by a chain of substitutions, possibly interleaved with factoring. Also, it should be noted that the completeness of this approach is proved only in the linear/affine case.

Termination is only argued informally, not proved. Each general substitution retires its random into *used* and is never repeated for it, bounding the number of general steps by $|\mathcal{R}|$; between general steps, simple substitutions strictly reduce the candidate pool; factoring is retried at most once per stall before the engine gives up on the current form. Interleaving factoring with two mutually triggering find-rules resists a clean potential-function argument, so a generous fuel counter backstops the

loop instead. Exhausting fuel is treated as failure, which is sound (Theorem 5.1 is never invoked on a fuel-out) but not necessarily tight: a fuel-out reports the probe as though it were genuinely insecure, without exhibiting an actual distinguishing observation.

5.2.3 Relation to the Coupling-Construction View

A discharge does more than answer SECURE/INSECURE. Read the right way, it hands back the object Chapter 4 asks for: a coupling for the probe. We state that object declaratively first, then show how the engine produces one.

We let the secrets range, rather than staying fixed as in Section 2.4. Write $\mathbf{w}(\rho;\mathbf{a})$ for the value of the probe under randomness assignment $\rho \in \{0,1\}^{\mathcal{R}}$ and secret assignment $\mathbf{a} \in \{0,1\}^{\mathcal{S}}$, the public inputs are held fixed.

Definition 5.1 (Coupling of a probe)**.** *A* coupling *of a probe* $\mathbf{w}$ *is a family* $\{\Phi_{\mathbf{a},\mathbf{b}}\}_{\mathbf{a},\mathbf{b}\in\{0,1\}^{\mathcal{S}}}$ *of measure-preserving bijections of* $\{0,1\}^{\mathcal{R}}$*, one per ordered pair of secret assignments, satisfying*

$$\mathbf{w}(\rho;\mathbf{a}) \;=\; \mathbf{w}\Big(\Phi_{\mathbf{a},\mathbf{b}}(\rho);\,\mathbf{b}\Big) \qquad \textit{for every } \rho \in \{0,1\}^{\mathcal{R}}.$$

Definition 5.1 says precisely where to send the randomness so that a run under $\mathbf{a}$ is reproduced, observation for observation, by a run under $\mathbf{b}$. Each $\Phi_{\mathbf{a},\mathbf{b}}$ dictates the relabelling of the random variables that absorb the change of secrets. Since it is measure-preserving, pushing the uniform randoms through it turns this pointwise identity into an equality of laws, so the distribution of $\mathbf{w}(\cdot;\mathbf{a})$ matches that of $\mathbf{w}(\cdot;\mathbf{b})$. A coupling existing for *every* pair $(\mathbf{a},\mathbf{b})$ is thus the same statement as $\mathbf{w}$ being secret-independent — which is what makes it the right certificate. The family carries the structure of a groupoid: one may take $\Phi_{\mathbf{a},\mathbf{a}} = \mathrm{id}$, $\Phi_{\mathbf{b},\mathbf{a}} = \Phi_{\mathbf{a},\mathbf{b}}^{-1}$, and $\Phi_{\mathbf{b},\mathbf{c}} \circ \Phi_{\mathbf{a},\mathbf{b}} = \Phi_{\mathbf{a},\mathbf{c}}$, so for any secret assignment $\mathbf{a}_0$, $\{\Phi_{\mathbf{a}_0,\cdot}\}$ already generate the rest of the couplings. It is these properties we care about, not any one procedure that realises them.

What the engine records is actually such a coupling. Discharging a probe amounts to finding a *derivation* of a finite sequence of rewrites

$$\sigma_1 = (r_1 \mapsto e_1 \oplus r_1),\ \ldots,\ \sigma_K = (r_K \mapsto e_K \oplus r_K),$$

where $r_i \notin \mathrm{vars}(e_i)$. And after the rewrites no secret variable remain. It does not

matter which rule proposed it, and whether the random it retires occurred once or many times, each σ_i is a substitution in the sense of Definition 2.3, hence also a measure-preserving shear $\widehat{\sigma_i}$ (Lemma 2.1). The coupling is assembled from these shears and from nothing else, so the construction below never needs to know how any individual rewrite was found.

Theorem 5.1 (Soundness of discharge)**.** *If* CheckProbe($\mathbf{w}$) *(Algorithm 1) returns* Secure*, then the joint distribution of* $\mathbf{w}(\rho;\mathbf{a})$*, for* ρ *sampled uniformly at random, does not depend on the secret assignment* $\mathbf{a}$*. Moreover the recorded substitutions realise a coupling of* $\mathbf{w}$ *in the sense of Definition 5.1.*

Proof. Fix a secret assignment $\mathbf{a}$. Each recorded rewrite step contributes the shear $\widehat{\sigma_i}$, whose defining context e_i may itself contain secrets, so write $\widehat{\sigma_i}^{\mathbf{a}}$ for its action under $\mathbf{a}$ and set

$$\varphi_{\mathbf{a}} \;=\; \widehat{\sigma_1}^{\mathbf{a}} \circ \widehat{\sigma_2}^{\mathbf{a}} \circ \cdots \circ \widehat{\sigma_K}^{\mathbf{a}},$$

which is a measure-preserving bijection of $\{0,1\}^{\mathcal{R}}$. It is the "rewrite recipe" instantiated at $\mathbf{a}$.

Also let $\overline{\mathbf{w}} = \mathbf{w}\sigma_1 \cdots \sigma_K$ be the probe state the derivation ends on. Applying Lemma 2.1 once per step and composing gives, for every ρ,

$$\mathbf{w}\Big(\varphi_{\mathbf{a}}(\rho);\mathbf{a}\Big) \;=\; \overline{\mathbf{w}}(\rho).$$

A Secure verdict means $\overline{\mathbf{w}}$ is syntactically secret-free, so its value is one and the same for every secret assignment. Since $\varphi_{\mathbf{a}}$ preserves the uniform law on $\{0,1\}^{\mathcal{R}}$, the normalisation identity makes the law of $\rho \mapsto \mathbf{w}(\rho;\mathbf{a})$ equal to the law of $\overline{\mathbf{w}}$, which does not depend on $\mathbf{a}$. Hence the joint distribution of the probe does not depend on the secret assignment.

For the coupling consider any two secret assignments $\mathbf{a},\mathbf{b}$. The normalisation identity sends both to the common normal form,

$$\mathbf{w}\Big(\varphi_{\mathbf{a}}(\rho);\mathbf{a}\Big) \;=\; \overline{\mathbf{w}}(\rho) \;=\; \mathbf{w}\Big(\varphi_{\mathbf{b}}(\rho);\mathbf{b}\Big).$$

Substituting $\rho \mapsto \varphi_{\mathbf{a}}^{-1}(\rho)$ and setting $\Phi_{\mathbf{a},\mathbf{b}} = \varphi_{\mathbf{b}} \circ \varphi_{\mathbf{a}}^{-1}$ yields $\mathbf{w}(\rho;\mathbf{a}) = \mathbf{w}\Big(\Phi_{\mathbf{a},\mathbf{b}}(\rho);\mathbf{b}\Big)$ for all ρ. Each $\Phi_{\mathbf{a},\mathbf{b}}$ is a composition of measure-preserving bijections, so the family $\{\Phi_{\mathbf{a},\mathbf{b}}\}$ is the sought after coupling of $\mathbf{w}$. □

5.3 Component II: Traversing the Subset Space

d-probing security asks that *every* set of at most d wires be secret-independent (Section 2.3.2). Read literally, that is $\sum_{i=1}^{d} \binom{N}{i}$ separate questions for a circuit of N wires. Component II is the bookkeeping that keeps the question exact while never enumerating a subset redundantly.

Two elementary observations are helpful to understand the traversal. First, we should see that secret-independence passes to subsets, specifically a subset of a secret-independent set is a marginal of a secret-independent law, hence itself is secret-independent. Certifying a set of wires therefore certifies all of its subsets in one stroke. Second, it is enough to look at probes of size *exactly* d: a smaller probe is a marginal of some size-d probe (whenever $N \geq d$, which always holds here), so it is covered once the size-d probes are. The task is thus to certify every d-element subset of the wire set, in batches as large as possible rather than one at a time.

5.3.1 The Representation of the Subset Space

The traversal never holds an explicit list of subsets; it holds a compact description of a whole family of them at once. A *factor* is a pair (c, W) of a count $c \in \mathbb{N}$ and a wire set W, standing for the family $\binom{W}{c}$ of all c-element subsets of W. A *worklist* is a finite sequence of factors

$$wl = \Big\langle (c_1, W_1), \ldots, (c_m, W_m) \Big\rangle, \qquad W_1, \ldots, W_m \text{ pairwise disjoint},$$

denoting the probes obtained by drawing c_j wires from each W_j and taking the union:

$$[\![wl]\!] \;=\; \Big\{ S_1 \cup \cdots \cup S_m \;:\; S_j \in \tbinom{W_j}{c_j} \Big\}, \qquad \big|[\![wl]\!]\big| \;=\; \prod_{j=1}^{m} \binom{|W_j|}{c_j}.$$

Disjointness of the W_j makes that union a disjoint one, so every probe has size $\sum_j c_j$ and none is described twice. Two vacaous worklists items are worth mentioning: if some factor asks for more than it holds, $|W_j| < c_j$, then $\binom{W_j}{c_j} = \emptyset$ and $[\![wl]\!]$ is empty; if every $c_j = 0$, the sole member is the empty probe. Either way there is nothing to certify. The verification task itself is the single-factor worklist $\big\langle (d, \mathcal{W}) \big\rangle$, whose

denotation is $\binom{\mathcal{W}}{d}$, i.e. all d-subsets of the circuit's wires $\mathcal{W}$.

Algorithm 2 sweeps this representation with three moves: discharge the union of the whole worklist and, on success, prune the branch; otherwise certify a representative probe and extend it to a safe set; then split each factor about that safe set and recurse on the pieces. The next two subsections treat the moves prune-and-extend and split, and presents a proof that the split loses nothing. In this section the extension part is kept ambigious, as its inner workings will be descredin in Section 5.4.

Algorithm 2 Traversing the subset space (Component II)

function CHECKALL($wl = \langle (c_1, W_1), \ldots, (c_m, W_m) \rangle$)
  if $[\![wl]\!] = \emptyset$ **then return** SECURE ▷ some $|W_j| < c_j$, or every $c_j = 0$
  end if
  $U \leftarrow W_1 \cup \cdots \cup W_m$
  if CHECKPROBE(U) = SECURE **then return** SECURE ▷ large-set pruning (§5.3.2)
  end if
  $\mathbf{w} \leftarrow$ a representative of $[\![wl]\!]$ ▷ c_j wires from each W_j
  $(sec, T) \leftarrow$ CHECKPROBE($\mathbf{w}$) ▷ Component I (§5.2)
  if not *sec* **then return** INSECURE($\mathbf{w}$)
  end if
  $\mathsf{safe} \leftarrow$ EXTEND($\mathbf{w}, T, U$) ▷ Component III (§5.4); $\mathbf{w} \subseteq \mathsf{safe} \subseteq U$
  for all $(i_1, \ldots, i_m) \in \prod_j \{0, \ldots, c_j\}$ **with** $(i_j)_j \neq (c_j)_j$ **do**
    $wl' \leftarrow \left\langle (i_j,\ W_j \cap \mathsf{safe}),\ \ (c_j - i_j,\ W_j \setminus \mathsf{safe}) \right\rangle_{j=1}^{m}$ ▷ drop zero-count parts
    $r \leftarrow$ CHECKALL(wl')
    if $r \neq$ SECURE **then return** r
    end if
  end for
  return SECURE
end function

5.3.2 Large-Set Pruning

Before splitting anything, the traversal tries to discharge the whole batch at once. Let $U = W_1 \cup \cdots \cup W_m$ be every wire the worklist can reach, and hand U — as a single large probe — to the rewrite engine of Component I. If U is certified secret-independent, so is every subset of U, and in particular every probe in $[\![wl]\!]$; the branch is secure and the traversal returns without descending. On some secure circuits this one test does almost all of the work, some branches never split, because their union already discharges.

However this discharge may fail, and it is worth seeing why the algorithm cannot stop here on failure. The union U is typically far larger than d, and a large tuple can be secret-*dependent* while every one of its size-d sub-tuples is secret-independent, hence failing to discharge U says nothing about the individual probes. When the union fails, we descend. We pick a representative $\mathbf{w} \in [\![wl]\!]$ — one probe of the target size, c_j wires from each factor — certify it with Component I, and, if it holds, grow it with Component III (Section 5.4) into a *safe set*

$$\mathsf{safe} \supseteq \mathbf{w}, \qquad \text{every subset of which is secret-independent.}$$

This will be further explained in Component III. Put briefly, one measure-preserving coupling blinds every wire of safe simultaneously, so any subset of safe, of any size, is secret-independent. Because the representative draws at least one wire from every factor, safe meets every W_j. This single fact is what makes the split below both exhaustive and strictly progressing.

5.3.3 Completeness via Vandermonde's Identity

With a safe set in hand the split is forced by a counting identity. Fix safe and cut each factor's ground set along it,

$$W_j = A_j \uplus B_j, \qquad A_j = W_j \cap \mathsf{safe}, \quad B_j = W_j \setminus \mathsf{safe}.$$

Lemma 5.1 (Set Vandermonde)**.** *For disjoint wire sets A, B and any $c \geq 0$,*

$$\binom{A \uplus B}{c} = \biguplus_{i=0}^{c} \Big\{ S_A \cup S_B \,:\, S_A \in \tbinom{A}{i},\ S_B \in \tbinom{B}{c-i} \Big\},$$

a disjoint *union: each c-subset S lies in exactly the block $i = |S \cap A|$. Counting both sides recovers Vandermonde's identity, $\binom{|A|+|B|}{c} = \sum_{i=0}^{c} \binom{|A|}{i}\binom{|B|}{c-i}$.*

The lemma is immediate, a c-subset is determined by its safe part and its unsafe part, chosen independently. Stating it set-wise, and not merely as the numeric identity, is the trick. It says not only how many probes each block holds but exactly which. Applying it in every factor and multiplying out,

$$[\![wl]\!] = \prod_{j=1}^{m} \binom{W_j}{c_j} = \biguplus_{(i_1,\ldots,i_m)} [\![wl_{(i_j)}]\!], \qquad wl_{(i_j)} = \Big\langle (i_j, A_j),\ (c_j - i_j, B_j) \Big\rangle_{j=1}^{m},$$

one *cell* for each distribution $(i_1, \ldots, i_m) \in \prod_j \{0, \ldots, c_j\}$ recording, factor by factor, how many of a probe's wires come from the safe side. The cells are disjoint and exhaust $[\![wl]\!]$, since every probe declares in each factor exactly how much of it is safe and each cell is again the denotation of a worklist, on which the traversal recurses directly.

One cell needs no recursion however. The all-safe cell $(i_j)_j = (c_j)_j$ collects the probes drawn entirely from the safe side $A_j \subseteq \mathsf{safe}$ and every one of those is secret-independent by the guarantee of Component III. The algorithm skips it and recurses on the rest. Skipping that particular cell means, with each recursion we are strictly decreasing the amount of probe sets we have to consider, which gives us our termination guarantee.

Theorem 5.2 (Soundness of the traversal)**.** CheckAll(wl) *(Algorithm 2) terminates, and if it returns* Secure *then every probe in* $[\![wl]\!]$ *is secret-independent. In particular* $\textsc{CheckAll}\big\langle(d, \mathcal{W})\big\rangle$*, returning* Secure*, certifies d-probing security.*

Proof. Termination. Order worklists by the multiset $\{|W_1|, \ldots, |W_m|\}$ of their ground-set sizes, under the multiset order. Take any cell the algorithm recurses on: it is neither the all-safe cell nor empty, so some factor j^* has $i_{j^*} < c_{j^*}$, forcing $B_{j^*} \neq \emptyset$; and $A_{j^*} \supseteq \mathbf{w} \cap W_{j^*} \neq \emptyset$ because the representative meets every factor. So W_{j^*} is replaced by the two strictly smaller ground sets A_{j^*}, B_{j^*}, while no other factor's ground set grows ($A_j, B_j \subseteq W_j$). The multiset strictly decreases, so the recursion is well-founded.

Soundness, by induction along that order. CheckAll returns Secure only in one of three ways. If the union $U = \bigcup_j W_j$ discharges, every subset of U (hence all of $[\![wl]\!]$) is secret-independent by marginalisation. If a degenerate worklist bottoms out, $[\![wl]\!]$ is empty or the lone empty probe, which is trivially secure. Otherwise the representative certifies and the recursive cells all return Secure. By Lemma 5.1 those cells together with the skipped all-safe cell partition $[\![wl]\!]$; the all-safe cell is secret-independent by Component III, and each recursive cell is $[\![wl_{(i_j)}]\!]$ of strictly smaller measure, hence secret-independent by the induction hypothesis. Every probe of $[\![wl]\!]$ lies in exactly one part, so all are secret-independent. Finally, since $[\![\langle(d, \mathcal{W})\rangle]\!] = \binom{\mathcal{W}}{d}$, a Secure verdict on $\langle(d, \mathcal{W})\rangle$ certifies that every d-subset of $\mathcal{W}$ is secret-independent, i.e. d-probing secure. □

We close this section with two remarks. The soundness above is one-directional by design: a Secure verdict is trustworthy, but the traversal inherits Component I's incompleteness verbatim and nothing more. An Insecure verdict is raised only

when Component I declines a representative (Section 5.2.2), never because a subset slipped through unexamined. Lemma 5.1 guarantees the cells cover $[\![wl]\!]$ exactly, so the sweep is genuinely exhaustive. The reported witness is therefore a true counterexample precisely when Component I is complete on it, as in the linear/affine case; a fuel-bounded refusal there is the one place a spurious INSECURE can enter, and it enters through Component I, not through this traversal. Second, the efficiency of the whole scheme rests on the safe set being large: the bigger safe, the fewer and smaller the surviving cells, and the closer the union prune comes to settling a branch outright. That is exactly the quantity Component III's coupling extension is built to maximise, and it is where the approach of this thesis departs from a plain set-splitting search.

5.4 Component III: Extending Probe Sets

Component II leans on a technique which we are yet to discuss. Every time it certifies a representative probe **w**, it asks for a whole *safe set* of wires around **w**, every subset of which is again secret-independent, and it uses that set to skip the all-safe cell of its Vandermonde split (Section 5.3.3). Component III is all about that safe set. It takes the certified probe and grows it into such a safe set, reusing the work the rewrite engine has already spent on **w** instead of certifying each new wire from nothing. The larger the safe set it hands back, the more the traversal prunes, so the whole scheme is only as efficient as this component is generous.

The three mechanisms we describe share one shape. Each of them walks over the wires of the circuit that are not yet in the safe set and decides, for a candidate u, whether the set stays jointly secret-independent once u is added. What separates them is the certificate they reuse and the test they run on it. Their soundness rests on a single fact which is the following: suppose φ is a measure-preserving bijection of the randomness space, and suppose that for every wire u in a set U the composite $u \circ \varphi$ is independent of the secrets. Since φ preserves the uniform law, the joint distribution of the wires of U agrees with the joint distribution of the composites $u \circ \varphi$, and the latter does not move with the secrets. So the joint law of U does not vary with the secrets, and by marginalisation neither does the law of any subset of U. A single measure-preserving map that blinds every wire of U at once thus certifies the entire set. We should add that the certified probe **w** is always kept in the safe set without a fresh test of course; the per-wire tests below are run only on

the other candidates. We present the three mechanisms from the weakest to the strongest and adopt the last.

5.4.1 Closure-Driven Extension

The first mechanism is set apart from the other two by *when* it runs. A closure is not computed after a probe has been certified. It is grown together with the probe, in the same topological pass that assembles it. As the traversal walks the circuit and picks wires to fill the chosen probe of the current worklist, it keeps a running set of the wires already forced by the ones picked so far, and each new choice drains that set forward. Two local rules drive the drain. A gate all of whose inputs are already known becomes known in turn, since its value is then fixed, and when a known wire is an $\oplus$ of inputs of which all but one are known, that last input becomes known as well, recovered as the sum of the wire and the others. The pass finishes with the chosen probe and, around it, every wire these two rules could deduce from it.

Everything the drain adds is a deterministic function of the probe. Writing $\mathbf{w}$ for the chosen probe and u for a deduced wire, we have $u = f(\mathbf{w})$ for the function f that the rules compose, so u carries no randomness the probe does not already carry, i.e. $H(u \mid \mathbf{w}) = 0$. Adjoining it leaves the joint entropy untouched, $H(\mathbf{w} \cup \{u\}) = H(\mathbf{w})$, and the pair $(\mathbf{w}, u)$ is a deterministic image of $\mathbf{w}$. Its dependence on the secrets is therefore exactly the probe's, which the engine has already certified to be none. That is the whole of the closure argument, and it needs no coupling.

What the closure buys is little. In a masked circuit almost no wire is a plain function of a handful of others, so the set deduced around a small probe is rarely more than a few relabellings, and the traversal is left to split nearly the whole space by hand. The shortfall sharpens as Component II recurses. Its worklist factors draw from wire sets carved out of the safe and unsafe splits, so deep in the recursion the candidate pool is a scattering of wires gathered from distant, unrelated parts of the circuit. Closure deduces a wire only from its immediate neighbours, and those neighbours become rare in a fragmented pool, so the deduction finds little to do exactly where the traversal needs the help most. In our experiments this mechanism extended probes the least of the three, although it is the cheapest of them, since the deduction rides along inside a pass the traversal was making anyway and never calls the engine a second time. Its worth is mostly conceptual.

5.4.2 Replay-Driven Extension

A stronger idea, due to Barthe et al. (Barthe et al., 2019), is to reuse the derivation rather than the probe. Certifying $\mathbf{w}$ left a short record of the steps that discharged it (Section 5.2.3), and each step is an operation one can carry out on any expression, not only on $\mathbf{w}$. Replay-driven extension takes a candidate wire u from the pool of the current worklist and runs that same record on it, step by step. If u comes out free of secrets at the end, then it is blinded by the very derivation that blinded the probe, and it joins the safe set. This reaches well past the closure, because a wire no longer has to be a function of the probe to survive the replay. It only has to be carried to a secret-free form by the same steps, which many of the sibling wires of a masked gadget are.

Barthe et al.'s derivations are built from two kinds of step. The first is the substitution of an invertible random by its context, the optimistic-sampling move our own engine makes. The second is an algebraic normalisation, which rewrites an expression into its algebraic normal form (its unique multilinear $\mathbb{F}_2$ polynomial, Section 2.1.1) and so lets products cancel. Barthe turns to it only when no substitution applies, and records it in the derivation like any other step. When the replay meets such a step it normalises the candidate too.

For us replays are a little problematic. Our engine does not lean on normalisation to free a buried random. It factors the expression instead (Section 6.1.3), reshaping a product so that a random trapped inside it moves into an additive position where a substitution can reach it. Factoring wins the engine circuits that substitution and normalisation alone would miss, but it is not an easily replayble step. Re-applying a factoring to an unrelated candidate wire does not carry over the blinding the way re-applying a substitution does, so the record we keep holds only the substitutions and leaves the factoring implicit. Replaying that record on a candidate is then incomplete. A wire the derivation genuinely blinds can fail it, because the cancellation that clears its secret was arranged by a factoring the replay never performs. That mismatch is the whole reason why we take one further step, and read the derivation not as a script to rerun but as a coupling function.

5.4.3 Coupling-Driven Extension

The mechanism we adopt keeps replay's use of the derivation and replaces its final test with an exact one. Section 5.2.3 already read the recorded derivation as a single measure-preserving bijection φ of the randomness space, the Φ that certified $\mathbf{w}$. Coupling-driven extension admits a candidate wire u exactly when $u \circ \varphi$ is independent of the secrets, and it decides that independence semantically rather than by surface syntax. Every wire replay admits is admitted here too, and so are the wires replay had to withhold because a factoring had to set up cancellations.

The candidates are not all the wires of the circuit. They are the wires of the current worklist, the pool C that Component II is trying to certify (Section 5.3), and the probe $\mathbf{w}$ is kept among them with no test at all. To decide $u \circ \varphi$ is secret-free exactly, for every candidate, is not cheap, so the work is staged. Initially, a fast probabilistic filter throws out the candidate wires that are incompatible with the coupling function we derived via evaluations.

Algorithm 3 Coupling-driven extension (Component III)

function EXTEND($\mathbf{w}, \varphi, C$)
Require: certified probe $\mathbf{w}$, coupling φ, candidate pool C from the worklist
Ensure: safe set safe with $\mathbf{w} \subseteq \mathsf{safe} \subseteq C$, with every subset secret-independent
    $\mathsf{safe} \leftarrow \mathbf{w}$
    for all $u \in C \setminus \mathbf{w}$ **do**
        if FASTREJECT(u, φ) **then**
            continue ▷ Phase 1: probabilistic filter, rejects proven-dependent u only
        end if
        if SURFACEFREE(u, φ) **or** ANFFREE(u, φ) **then** ▷ Phases 2–3: exact, decides $u \circ \varphi$ secret-free
            $\mathsf{safe} \leftarrow \mathsf{safe} \cup \{u\}$
        end if
    end for
    return safe
end function

The filter is an evaluation under two runs. Fix random values for the randoms, evaluate $u \circ \varphi$ once with the secrets also drawn at random and once with the secrets set to zero, and compare. If the two disagree, then $u \circ \varphi$ genuinely moves with the secrets, and u is dropped. Agreement proves nothing on its own, since a dependence can hide from any one tape, so a survivor is not yet admitted. Running the trial over many tapes makes a surviving dependence steadily less likely but never impossible, which is why the filter is allowed to reject but never to accept.

Then the derivation is replayed on u, and the result is first read by ordinary $\mathbb{F}_2$ cancellation, which is cheap and clears the common case. When a secret still shows, the wire is converted to its algebraic normal form and admitted exactly when no secret variable survives there. The probabilistic filter and this exact stage differ in kind, so it is worth recording that together they never admit a leaking wire and never turn away a blinded one.

Lemma 5.2 (The extension test is exact)**.** *Extend admits a candidate u if and only if $u \circ \varphi$ is independent of the secrets.*

Proof. Write $v = u \circ \varphi$. Its algebraic normal form (ANF) is the unique multilinear $\mathbb{F}_2$ polynomial computing v, so a variable is absent from that form exactly when v does not depend on it. In particular,

$$v \text{ is independent of the secrets} \iff \text{no secret variable occurs in the ANF of } v.$$

A wire is admitted only at the exact stage. Phase 2 accepts when the surface form of v carries no secret, and Phase 3 accepts when the ANF of v carries no secret variable. By the equivalence, each makes v independent of the secrets, so every admitted wire is truly blinded. Conversely, suppose v is independent of the secrets. With the randoms fixed, v is then constant in the secrets, so the filter's two evaluations agree on every tape and never reject u, and it reaches the exact stage. And because the ANF of v must not carry a secret variable, Phase 3 will admit it. Hence u is admitted if and only if v is independent of the secrets. □

Theorem 5.3 (Safe set)**.** *Let* $\mathsf{safe} = \textsc{Extend}(\mathbf{w}, \varphi, C)$ *for a probe* $\mathbf{w}$ *certified by Component I with coupling φ. Then the joint distribution of the wires of* safe*, over a uniformly random ρ, does not depend on the secret assignment, and in particular every subset of* safe *is secret-independent.*

Proof. By Lemma 5.2, safe consists of the probe $\mathbf{w}$ and the candidates u whose composite $u \circ \varphi$ is independent of the secrets. Since that composite does not depend on the secret assignment, it is a function of the randomness alone; so simply write its value as $\overline{u}(\rho)$.

Fix a secret assignment $\mathbf{a}$ and let $\varphi_{\mathbf{a}}$ be the induced measure-preserving bijection of the randomness space (Section 5.2.3). Chaining Lemma 2.1 through the recorded steps gives, for every $u \in \mathsf{safe}$ and every ρ,

$$u\Big(\varphi_{\mathbf{a}}(\rho); \mathbf{a}\Big) \;=\; (u \circ \varphi)(\rho; \mathbf{a}) \;=\; \overline{u}(\rho).$$

The left-hand side is u evaluated at the relabelled randomness assignment $\varphi_{\mathbf{a}}(\rho)$ under the secrets $\mathbf{a}$, so it is a bit value; the first equality is the substitution lemma, the second one comes from the admission condition. Both copies of $\mathbf{a}$, the one inside $\varphi_{\mathbf{a}}$ and the one feeding u its secret inputs, cancel, and the value depends on ρ alone.

Now hold $\mathbf{a}$ fixed and draw ρ uniformly. As $\varphi_{\mathbf{a}}$ preserves the uniform law, $\varphi_{\mathbf{a}}(\rho)$ is uniform too, so the joint law of the safe wires under $\mathbf{a}$, that of $\big(u(\rho;\mathbf{a})\big)_{u\in\mathsf{safe}}$, equals the joint law of $\big(u(\varphi_{\mathbf{a}}(\rho);\mathbf{a})\big)_u = \big(\overline{u}(\rho)\big)_u$. This last family depends on no secret, so the joint law is one and the same for every $\mathbf{a}$. Hence the distribution of safe does not depend on the secret assignment, nor any of its subsets. □

Finally we observe that the other two mechanisms admit nothing if this one does not.

Proposition 5.1 (Closure and replay sit inside the coupling extension)**.** *Fix a certified probe* $\mathbf{w}$ *with coupling* φ*, and write* $\mathsf{safe}_{\mathrm{clo}}$*,* $\mathsf{safe}_{\mathrm{rep}}$*, and* $\mathsf{safe}_{\mathrm{cou}}$ *for the wires admitted by the closure, replay, and coupling extensions. Then* $\mathsf{safe}_{\mathrm{clo}} \subseteq \mathsf{safe}_{\mathrm{cou}}$ *and* $\mathsf{safe}_{\mathrm{rep}} \subseteq \mathsf{safe}_{\mathrm{cou}}$.

Proof. For replay, a wire is admitted only after its replay rechecks to a secret-free form, which means $u\circ\varphi$ is independent of the secrets; the coupling test admits every such wire by Lemma 5.2. For the closure, an admitted wire is a gate-composition $u = f(\mathbf{w})$. Evaluation respects that composition, so $u\circ\varphi = f(\mathbf{w}\circ\varphi)$ as functions of the randomness; the coupling leaves $\mathbf{w}\circ\varphi$ independent of the secrets, and a gate-composition of functions independent of the secrets is again independent of them, so $u\circ\varphi$ is too and the coupling test admits u. □

6. IMPLEMENTATION OF COPPULA

6.1 Expression Representation

The rewrite engine of Component I (Section 5.2) works entirely on the expressions of a probe. It looks for a random in an additive context, substitutes it by its context, and asks whether any secret is still in sight. Every one of these moves is an operation on the *form* of the expression, so the structure that holds that form is what decides how cheap the engine can be. This section describes that structure. It is a hash-consed n-ary DAG over $\mathbb{F}_2$, wrapped in a layer of smart constructors that keep every expression canonical, and extended with a multiplicative factoring pass that reshapes products so the engine can reach randoms otherwise buried inside them.

One point of clarification before we dive in. The circuit of Definition 2.1 is strictly 2-ary, and that is deliberate. Each 2-input gate output is a distinct observable, so collapsing gates would hide intermediate wires the adversary is allowed to probe (Section 2.3.2). The representation here is not the circuit, and this important. It is how we store the *value* of a wire for the algebra, and there flattening a chain of same-typed gates into one n-ary node loses nothing, since only the wire's function matters and not its gate-by-gate history. So the two structures (one faitfhul to the topology and one keeping the expressions) live side by side. The 2-ary netlist stays the authority on what may be probed, and the DAG is the authority on what each probed wire computes.

6.1.1 A Hash-Consed n-Ary Circuit DAG

An expression is a node of one of four kinds. It is a *leaf* carrying an input variable (secret, random, or public), a *constant* 0 or 1, an n-ary sum $\oplus$ over a list of operand nodes, or an n-ary product $\odot$ over a list of operand nodes. A wire's value is the node its defining sub-circuit builds, and the whole circuit is a single DAG in which wires that share sub-structure share nodes.

Sharing is the first reason for the design, and it is not incidental. A masked gadget reuses the same handful of masks and shares across many gates, so the naive syntax tree of a probe would carry the same subexpression over and over again. We store each distinct node once. A table, keyed by a node's kind together with its operand list, which maps a structure to the identifier of the node already built for it; this is *hash-consing*. Constructing a node is then a lookup in that table. On a hit we return the existing identifier, and on a miss we allocate a fresh one and record it. Two expressions that are structurally the same are therefore the same node, physically, however many times they are written.

The second reason is equality. Because interning makes structural identity coincide with identifier identity, deciding whether two expressions are equal is a comparison of two integers, with no traversal at all. The rewrite engine leans on this everywhere. Recognising that a substitution has made two operands coincide so that they cancel, checking whether a rewritten probe matches one already seen, and, in Component III, testing a candidate wire's coupling image against a secret-free target, are each an $O(1)$ test on identifiers rather than a walk over two trees. We should note that this fast equality is only as meaningful as the expressions are canonical: hash-consing alone shares expressions written identically, and it is the next subsection that makes identical identifiers mean equal values. We also keep the identifiers of the random and secret leaves in their own lists, so iterating over vars($\mathbf{w}$), or scanning for a surviving secret, never has to walk the entire DAG.

6.1.2 Canonicalization: Flattening and Cancellation

Hash-consing shares expressions that are written identically, which on its own is weak. In $\mathbb{F}_2$ the sums $a \oplus b$ and $b \oplus a$ are equal, yet written in that order they are two different nodes. We close this gap with a layer of *smart constructors*. Nothing

builds a sum or a product node directly; every sum is made through one constructor and every product through another, and these put the operands into a canonical form before interning. Two expressions with the same value then reach the same canonical node, so the fast equality of Section 6.1.1 becomes equality of value and not merely of syntax, at least for the identities we fold in, which are exactly the ones the engine needs.

Three normalisations do the work. The first is *flattening*. Associativity lets a sum of sums be one flat sum, and likewise a product of products be one flat product. This is where the n-ary shape earns itself, as a chain of 2-ary XOR gates from the netlist collapses into a single sum node holding all the leaves, and any nesting is quotiented away. The second is *ordering*. Commutativity lets us keep each operand list sorted by identifier, so the order of writing is irrelevant to identity and $a \oplus b$ and $b \oplus a$ intern to one node. The third folds in the $\mathbb{F}_2$ identities the engine's cancellations depend on. In a sum, the constant 0 is dropped as the additive identity, and a repeated operand is deleted in pairs, since $x \oplus x = 0$. In a product, the constant 1 is dropped as the multiplicative identity, the constant 0 collapses the whole product to 0 by annihilation, and a repeated operand is kept only once, since $x \odot x = x$ by idempotence (Section 2.1.1). Degenerate arities settle the same way. An empty sum is 0, an empty product is 1, and a one-operand sum or product is that operand.

The payoff is that the one algebraic move the whole rewrite method turns on happens for free. When the engine substitutes a single-occurrence random r by its context, replacing the parent $x = e \oplus r$, the cancellation $e \oplus (e \oplus r) = r$ that Corollary 2.1 promises is not a step the engine performs. It re-interns the rewritten expression through the sum constructor, the two copies of e meet in the sorted operand list and annihilate, and the canonical result is already the collapsed form. The idempotent and annihilating collapses that a general substitution triggers happen the same way. The engine proposes rewrites, and the constructors do the algebra.

6.1.3 Multiplicative Factoring

Canonicalisation normalises the additive structure of an expression, but it leaves products opaque. The find-rules of Component I (Sections 5.2.1 and 5.2.2) look for a random in *additive* position, as an operand of a sum, because that is the shape Lemma 2.1 rewrites. A random that occurs only inside products is invisible to them, even when the expression could in fact be discharged. Multiplicative factoring is

the pass that reshapes such an expression. It pulls a shared factor out of a sum of products so that a trapped random surfaces additively, where a substitution can reach it. It is the second tier of the engine and is interleaved into the third (Chapter 5).

6.1.3.1 The Factoring Transformation

Factoring applies the distributive law right to left, over a sum whose operands are products. If a node f appears as an operand in two or more of those products, we pull it out,

$$(f \odot a) \oplus (f \odot b) \oplus \mathit{rest} = \Big(f \odot (a \oplus b)\Big) \oplus \mathit{rest},$$

where *rest* are the summands not carrying f, and a, b are what remains of the two products once f is removed. Over $\mathbb{F}_2$ this is an identity, so the wire's value is untouched and only its form changes.

What we pull out is chosen greedily. Over the operands of a sum we count, for each candidate factor, in how many of the product summands it appears, and take the one with the highest count, provided it appears at least twice, since a factor sitting in a single product buys nothing. Ties are broken by identifier, for determinism. With the factor f fixed, we partition the summands into those carrying f and those not, strip f from each of the former, sum the remainders into an inner sum, multiply that by f, and add the untouched summands back, every construction going through the canonicalising constructors of Section 6.1.2. Factoring is then applied recursively, bottom up over the DAG and repeated on its own result, until no sum has a factor of multiplicity two or more.

For a concrete instance we take a probe from the first-order masked quadratic bijection Q_{12}^4, one of the gadgets we verify. Its inputs are shared as $a_1 = a \oplus r_0$, $b_1 = b \oplus r_1$, $c_1 = c \oplus r_2$ together with the mask shares $b_2 = r_1$ and $c_2 = r_2$, where a, b, c are secret and r_0, r_1, r_2 are fresh randoms. One of the gadget's recombination wires gathers five product and share terms, and after canonicalisation its value is

$$w = (a \oplus r_0) \odot (b \oplus r_1) \oplus (a \oplus r_0) \odot (c \oplus r_2) \oplus (c \oplus r_2) \oplus (a \oplus r_0) \odot r_1 \oplus (a \oplus r_0) \odot r_2.$$

As it stands the simple rule cannot finish. It clears r_0, whose only occurrence is the additive one in $a \oplus r_0$, but b and c then remain, and neither r_1 nor r_2 is a single-

occurrence random. Each shows up both additively, in a share, and again inside the products, so the uniqueness the simple rule needs is absent and the general rule's side condition fails as well. Factoring is what breaks the deadlock. The factor $a \oplus r_0$ is shared by four of the five summands, and pulling it out drops their cofactors into a single inner sum,

$$w = (a \oplus r_0) \odot \Big[(b \oplus r_1) \oplus (c \oplus r_2) \oplus r_1 \oplus r_2\Big] \oplus (c \oplus r_2) = (a \oplus r_0) \odot (b \oplus c) \oplus c \oplus r_2,$$

where inside the bracket $r_1 \oplus r_1$ and $r_2 \oplus r_2$ cancel by the additive law of Section 6.1.2. The surplus copies of r_2 go with them, and what is left carries r_2 additively and exactly once. Now the single-occurrence pattern of Section 5.2.1 applies. The engine relabels the whole expression as a fresh random and reads w as secret-free.

6.1.3.2 Enlarging the Class of Verifiable Circuits

Factoring is sound by construction, being an $\mathbb{F}_2$ identity, so it never lets an insecure probe pass. What it changes is completeness, and it strictly enlarges the class of probes the engine can certify. Every probe the engine discharges without factoring it still discharges, since Algorithm 1 tries the unfactored form first and factors only on failure. And some probes, like the one above, are discharged only once factoring has exposed a random no additive rule could otherwise reach. The verifiable class with factoring is therefore a strict superset of the class without it.

Two cautions temper the picture, both settled already in Chapter 5. First, factoring is not monotone in the cheap direction. It can just as well destroy a single-occurrence opportunity that the unfactored form exposed, which is exactly why the engine tries the simple rule unfactored before it factors, rather than factoring once up front (Section 5.2.1). It has to be interleaved with substitution, not run as a preprocessing step. Second, factoring is not a replayable operation. Re-applying it to an unrelated candidate wire does not carry over the blinding the way re-applying a substitution does, and this is what forces Component III to read a derivation as a coupling with an exact endpoint test rather than as a script to rerun (Section 5.4.3). The gain in verifiable circuits is real, but it is bought with the extra machinery those sections build around it.

6.2 Implementing the Rewrite Engine

Algorithm 1 presents Component I as a single tiered procedure, but the tool realises its rule set twice, and the reason for the two is worth an explanation. One is a cheap, verdict-only path that answers secret-independence without ever rewriting the DAG, and the other is a complete path that rewrites for real and hands back the coupling. The tiering of CheckProbe threads them together. It tries the cheap path first on the unfactored probe, then once more after factoring, and only then falls through to the complete loop. The two paths answer different callers. Component II's large-set pruning (Section 5.3.2) throws the big union at the engine and wants only a yes or no, as fast as possible, whereas a certified representative must come back with the coupling that Component III will use (Section 5.4.3). The next two subsections take the paths in turn.

6.2.1 Reference-Counted Single-Occurrence Discharge

The cheap path applies the simple rule (Section 5.2.1) to a fixpoint without ever building a new node. It relies on the observation that the simple rule needs to know very little about a random. It needs how many times the random occurs and whether that single occurrence is additive, and it needs the parent to splice away. So instead of rewriting the expression the path keeps that bookkeeping and updates it in place.

A single depth-first pass over the probe's sub-DAG records, for every node, how many parents it has and how many of them are sums, together with its multiplicative depth. A random is a simple-rule candidate exactly when it has one parent and that parent is a sum, and it is not itself a probe root; the candidates are held in a todo list ordered by increasing multiplicative depth, so the shallowest gets rewritten first. Firing the rule on a random is then a local edit to the counts and not to the DAG. We mark its additive parent as freshly random, drop the random, and decrement the parent counts of the parent's other children. A sibling whose total count thereby falls to a single additive occurrence cascades back into the worklist as a new candidate, and it is this cascade that carries the fixpoint forward. The probe is secure once no secret is reachable from any root.

Keeping observed wires out of the todo list is an important soundness condition. A random that is itself a probe root must never be relabelled, since its value is exactly what the adversary sees, and marking its parent fresh while conditioning on it would be mathematically unsound. The cascade re-inserts a candidate only after the same root check.

What this path cannot do is produce a coupling. Its cascade steps fire on already-relabelled parents rather than on tape variables, so they are not the leaf relabels that Section 5.2.3 assembles a coupling from, and replaying them against another wire would diverge from the certified derivation. This costs nothing where the path is used. The large-set prune only needs the verdict, and a probe that must yield a coupling is sent to the complete loop instead anyways.

6.2.2 Recomputing the Find-Rules per Round

The complete loop (GeneralRewrite, Section 5.2.2) substitutes for real, as in it alters the DAG. It picks a random and an additive occurrence $x = e \oplus r$ and rewrites the probe by $r \mapsto e \oplus r$ throughout, re-interning the result through the canonicalising constructors of Section 6.1.2, which is what makes the chosen occurrence collapse to a bare r while every other absorbs e. Because the substitution genuinely reshapes the expression, the incremental counts of Section 6.2.1 cannot be carried across it. So the loop takes the opposite stance and recomputes. Each round runs one fresh depth-first pass over the current form, and that single pass does double duty. It answers whether any secret still survives, which is the loop's success test, and it produces the parent counts and depths the find-rules need. The pass is restricted to the randoms actually present in the probe rather than every random in the circuit.

On that snapshot, the loop first tries FindSimple, the same single-occurrence additive pattern as before, since a general substitution can turn a twice-occurring random into a once-occurring one; failing that, FindGeneral looks for a not-yet-used random with an additive parent whose other children do not mention it, which is the side condition Lemma 2.1 needs. That last check is a reachability question, does r occur anywhere below the context e, answered by a memoised walk whose memo is shared across the candidate's parents so the sub-DAG is not re-explored from scratch. Both rules prefer the shallowest candidate. When both stall the loop factors once and retries (Section 6.1.3), and a fuel counter backstops the whole thing, as Section 5.2.2 discussed.

The point of paying this recomputation is the record it leaves. Every substitution the loop commits is a leaf relabel $(r, e \oplus r)$, and appended in order these are exactly the shears $\sigma_1, \ldots, \sigma_K$ of Section 5.2.3. The loop hands that list back alongside its verdict, and it becomes the coupling Component III replays.

6.3 Evaluating the Coupling

Component III decides, for each candidate wire u, whether $u \circ \varphi$ is free of secrets, where φ is the coupling the engine just recorded (Section 5.4.3). Algorithm 3 gives that decision, together with a probabilistic filter first and an exact test after. Both stages need to evaluate a wire through the coupling, and this section describes how that evaluation is done cheaply.

6.3.1 Bit-Sliced Evaluation Through a Coupled Environment

The filter compares two runs of $u \circ \varphi$, one with the secrets drawn at random and one with them zeroed, and rejects u when they disagree (Section 5.4.3). A single pair of runs proves little, so the filter wants many, and the representation makes many nearly as cheap as one. Every wire value is carried as a machine word whose 64 bits are 64 independent $\mathbb{F}_2$ evaluations, one per lane. A sum is a word exclusive-or and a product a word and, so one traversal of the sub-DAG evaluates all 64 lanes at once, and node values are cached across the traversal so a shared subexpression is computed a single time.

Evaluating $u \circ \varphi$ never rebuilds u. Recall the coupling is a list of relabels $r \mapsto e \oplus r$; rather than substitute them into u, we precompute a *coupled environment* that binds each random to the value of its net image under the whole list and then evaluate u against it. Since a relabel's image may mention randoms that later relabels move, the environment is built by folding the list from right to left, so that by the time a random is bound, every relabel acting on its image is already computed. The identity we get is $\mathrm{eval}(u,\ env) = \mathrm{eval}(u \circ \varphi)$, so the two environments, secrets-as-random and secrets-as-zeros, are each built once and then reused for every candidate, with their evaluation caches shared across.

6.3.2 From Probabilistic Filter to Exact Verdict

Agreement across lanes is solid evidence but not a proof, so the filter is allowed to reject but never to accept (Lemma 5.2). A survivor has to be decided exactly. The coupling is applied to all survivors together, sharing one substitution memo per step rather than replaying the whole list per candidate, and each image is then read in two ways. First by ordinary $\mathbb{F}_2$ cancellation over the canonical form, which is cheap and clears the common case. When a secret still shows there, the image is converted to its algebraic normal form, and the wire is admitted exactly when no secret variable occurs in its ANF. The ANF is the canonical multilinear form (Section 2.1.1), so a secret is absent from it precisely when the wire does not depend on that secret; this is what makes the test exact rather than merely sound.

7. EVALUATION

7.1 Benchmarks

Describe the benchmark suite of masked circuits used to evaluate the tool (sources, sizes, masking orders).

7.2 Methodology and Metrics

Describe the evaluation methodology and the metrics reported (verification time, number of probes discharged, redundancy, etc.).

7.3 Results

Present the overall results of running the tool on the benchmark suite.

7.3.1 Verification Outcomes and Case Studies

Report verification outcomes per benchmark, including the case study of the insecure 4-share $x + yz$ finding.

7.3.2 Redundancy in the Search

Analyze redundancy in the search: union vs. chosen re-discharge, and closure collisions.

7.3.3 The Cost and Benefit of Extending Probe Sets

Measure the cost/benefit trade-off of Component III's probe-set extension against the baseline traversal.

7.4 Discussion

Discuss what the results imply about the tool's strengths, limitations, and how it compares to related work.

8. FUTURE WORK

8.1 Arithmetic Circuits on Larger Fields

Discuss extending the approach beyond $\mathbb{F}_2$ to larger fields.

8.2 Symmetry-Orbit Space Exploration

Discuss exploiting circuit automorphisms/relabeling symmetries to reduce the subset space explored.

8.3 Scaling and Compositional Verification

Discuss compositional verification techniques for scaling to larger circuits/gadgets.

8.4 Glitches and Practical Aspects

Discuss extending the model to account for glitches and other practical hardware effects.

9. CONCLUSION

Write the conclusion: summarize the problem, the approach (rewrite engine, subset-space traversal, probe-set extension), the main results, and their significance.

BIBLIOGRAPHY

Barthe, G., Belaïd, S., Cassiers, G., Fouque, P.-A., Grégoire, B., & Standaert, F.-X. (2019). Maskverif: Automated verification of higher-order masking in presence of physical defaults. In K. Sako, S. A. Schneider, & P. Y. A. Ryan (Eds.), *Computer security – esorics 2019* (pp. 300–318, Vol. 11735). Springer International Publishing. https://doi.org/10.1007/978-3-030-29959-0_15

Barthe, G., Belaïd, S., Dupressoir, F., Fouque, P.-A., Grégoire, B., & Strub, P.-Y. (2015). Verified proofs of higher-order masking. *Advances in Cryptology – EUROCRYPT 2015*, *9056*, 457–485. https://doi.org/10.1007/978-3-662-46800-5_18

Barthe, G., Grégoire, B., Hsu, J., & Strub, P.-Y. (2017). Coupling proofs are probabilistic product programs. *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages (POPL)*, 161–174. https://doi.org/10.1145/3009837.3009896

Chari, S., Jutla, C. S., Rao, J. R., & Rohatgi, P. (1999). Towards sound approaches to counteract power-analysis attacks. *Advances in Cryptology – CRYPTO '99*, *1666*, 398–412. https://doi.org/10.1007/3-540-48405-1_26

Ishai, Y., Sahai, A., & Wagner, D. (2003). Private circuits: Securing hardware against probing attacks. In D. Boneh (Ed.), *Advances in cryptology - crypto 2003* (pp. 463–481). Springer Berlin Heidelberg.

Kocher, P., Jaffe, J., & Jun, B. (1999). Differential power analysis. *Advances in Cryptology – CRYPTO '99*, *1666*, 388–397. https://doi.org/10.1007/3-540-48405-1_25

Rivain, M., & Prouff, E. (2010). Provably secure higher-order masking of AES. *Cryptographic Hardware and Embedded Systems – CHES 2010*, *6225*, 413–427. https://doi.org/10.1007/978-3-642-15031-9_28

Valiant, L. G. (1979). The complexity of computing the permanent. *Theoretical Computer Science*, *8*(2), 189–201. https://doi.org/10.1016/0304-3975(79)90044-6

Wagner, K. W. (1986). The complexity of combinatorial problems with succinct input representation. *Acta Informatica*, *23*(3), 325–356. https://doi.org/10.1007/BF00289117

APPENDIX

Add supplementary experimental data (extra tables, plots, benchmark details) supporting the Evaluation chapter.